Blatt 05: Interpreter

Carsten Gips, BC George (HSBI)

1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung eines Tree-Walking-Interpreter mit ANTLR für eine Lisp-artige Sprache.

2 Methodik

Sie finden im Sample Project zwei Grammatiken (MiniLispA, MiniLispB), die (teilweise) zu der Zielsprache auf diesem Blatt passen. Analysieren Sie beide Grammatiken und entscheiden Sie sich für eine der beiden Varianten. Vervollständigen Sie diese bzw. passen Sie diese an.

Erstellen Sie mit dieser Grammatik und ANTLR wieder einen Lexer und Parser.

Es ist empfehlenswert, den Interpreter dreistufig zu realisieren:

1. Einlesen aus einer Datei mit Lisp-Code und Parsen der Inhalte
2. Aufbauen der Symboltabelle und Durchführung der semantischen Analyse
3. Ablaufen des Parse-Tree/AST und Auswerten der Ausdrücke (Interpretation)

3 Sprachdefinition

Ein Programm besteht aus einem oder mehreren Ausdrücken (*Expressions*). Die Ausdrücke haben eine spezielle Form: Sie sind sogenannte S-Expressions. Dies sind entweder Literale der Form `x` oder einfache listenartige Gebilde der Form `(. x y)`, wobei der `.` eine Operation (oder Funktion) darstellt und `x` und `y` selbst wieder S-Expressions sind.

Die einfachste Form sind dabei Literale mit konkreten Werten der drei Datentypen `Integer`, `String` und `Boolean`:

```
42          ;; Integer
"hello"     ;; String
true        ;; Boolean
false       ;; Boolean
```

Für eine Variable `foo` wäre das Folgende ebenfalls eine S-Expression:

```
foo         ;; Variable foo
```

(Über `;;` wird ein Kommentar eingeleitet, der bis zum Ende der Zeile geht.)

Komplexere Ausdrücke werden über die Listenform gebildet:

```
(+ 1 1)                ;; 1 + 1
(/ 10 3)               ;; 10 / 3
(+ 1 2 3 4)            ;; 1 + 2 + 3 + 4
(+ (+ (+ 1 2) 3) 4)    ;; (((1 + 2) + 3) + 4)
(/ (+ 10 2) (+ 2 4))   ;; ((10 + 2) / (2 + 4))
```

In der listenartigen Form ist der erste Eintrag der Liste immer eine Operation (oder ein Funktionsname), danach kommen je nach Operation/Funktion (die Arität muss passen!) entsprechende Einträge, die als Parameter für die Operation oder Funktion zu verstehen sind.

Die Ausdrücke sind implizit von links nach rechts geklammert, d.h. der Ausdruck `(+ 1 2 3 4)` ist *syntactic sugar* für `(+ (+ (+ 1 2)3)4)`.

3.1 Eingebaute Funktionen

Es gibt zwei Funktionen, die fest in der Sprache integriert sind.

Mit der eingebauten Funktion `print` kann der Wert eines Ausdrucks auf der Konsole ausgegeben werden:

```
(print "hello world")
(print "wuppie\nfluppie\nfoo\nbar")
```

Die eingebaute Funktion `str` verknüpft ihre Argumente und bildet einen String. Falls nötig, werden die Argumente vorher in einen String umgewandelt.

```
(str 42)                              ;; liefert "42" zurück
(str "wuppie" "fluppie" "foo" "bar")  ;; liefert "wuppiefluppiefoobar" zurück
(str "one: " 1 ", two: " 2)           ;; liefert "one: 1, two: 2" zurück
```

3.2 Operatoren

Es gibt nur wenige vordefinierte Operatoren, diese mit der üblichen Semantik.

3.2.1 Vergleichsoperatoren

Operation	Operator
Gleichheit	=
Größer	>
Kleiner	<

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist immer vom Typ `Boolean`.

3.2.2 Arithmetische Operatoren

Operation	Operator
Addition	+
Subtraktion	–
Multiplikation	*
Division	/

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist vom Typ der Operanden.

3.3 Kontrollstrukturen (If-Else)

Die **`if`**`-then-`**`else`**-Abfrage gibt es mit und ohne den **`else`**-Zweig:

```
(if boolean-form
    then-form
    optional-else-form)
```

```
(if (< 1 2)
    (print "true")     ;; then
    (print "false"))   ;; else
```

Dabei kann jeweils nur genau eine S-Expression genutzt werden. Wenn man mehrere Dinge berechnen möchte, nutzt man **`do`**:

```
(do
    (print "wuppie")
    (print "fluppie")
    (print "foo")
    (print "bar"))
```

Beispiel:

```
(if (< 1 2) (do (print "true") (print "WUPPIE")) (print "false"))
```

oder anders formatiert:

```
(if (< 1 2)
    (do (print "true")
        (print "WUPPIE"))
    (print "false"))
```

3.4 Variablen: Bindings mit *def* anlegen

```
(def x 42)  ;; definiert eine neue Variable mit dem Namen "x" und dem Wert 42

x           ;; liefert 42
```

```
(+ x 7)      ;; liefert 49
```

3.5 Funktionen mit *defn* definieren

```
;;      name   params  body
(defn  hello  (n)     (str "hello " n))  ;; Definition einer Funktion "hello" mit einem Parameter

(hello "world")                          ;; Aufruf der Funktion "hello" mit dem Argument "world"
```

3.6 Lokale Scopes mit *let*

```
;;    bindings      use names here
(let  (name value)  (code that uses name))

(def x 99)   ;; globale Variable x
(def y 101)  ;; globale Variable y
(def z 42)   ;; globale Variable z
(let (x 1   ;; lokales x mit Wert 1(verdeckt globales x)
      y 2)  ;; lokales y mit Wert 2
     (+ x y z))  ;; 1+2+42

(defn  hello
       (n)
       (let (l 42)  ;; l is valid in this scope
            (str "hello " n ": " l)
       )  ;; end of local scope
)  ;; end of function definition
```

Mit `let` können lokale Variablen erzeugt werden, die dann in dem jeweiligen Scope genutzt werden können. Dies funktioniert wie in anderen Sprachen mit Scopes.

3.7 Rekursion

```
(defn fac (n)
    (if (< n 2)
        1
        (* n (fac (- n 1)))))
```

Da es kein **`while`** oder **`for`** gibt, müssen Schleifen über rekursive Aufrufe abgebildet werden.

3.8 Datenstrukturen

In unserer Sprache gibt es Listen:

```
(1 2 3)          ;; Fehler!
(def v (1 2 3))  ;; Fehler!
```

Das Problem daran ist, dass unsere S-Expressions zwar bereits listenartige Strukturen sind, der erste Eintrag aber als Operator oder Funktion interpretiert wird. Der Ausdruck oben würde beim Auswerten versuchen, die "Funktion" 1 auf den Argumenten 2 und 3 aufzurufen …

Man braucht also eine Notation, die ein sofortiges Auswerten verhindert und nur die Liste an sich zurückliefert. Dies erreicht man durch die Funktion `list`:

```
(list 1 2 3)          ;; (1 2 3)

(def v (list 1 2 3))  ;; v = (1 2 3)
v                     ;; (1 2 3)
```

Mit der Funktion `nth` kann man auf das n-te Element einer Liste zugreifen:

```
(nth (list "abc" false 99) 2)  ;; 99
```

Zusätzlich gibt es die beiden Funktionen `head` und `tail`, die das erste Element einer Liste bzw. die restliche Liste ohne das erste Element zurückliefern:

```
(head (list 1 2 3))  ;; 1
(tail (list 1 2 3))  ;; (2 3)
```

4 Aufgaben

4.1 A5.1: Grammatik und ANTLR (3P)

1. Erstellen Sie zunächst einige Programme in der Zielsprache. Diese sollten von einfachsten Ausdrücken bis hin zu komplexeren Programmen reichen. Definieren Sie beispielsweise eine Funktion, die rekursiv die Länge einer Liste berechnet.

   Definieren Sie neben gültigen Programmen auch solche, die in der semantischen Analyse zurückgewiesen werden sollten. Welche Fehlerkategorien könnte es hier geben?

2. Definieren Sie für die obige Sprache eine geeignete ANTLR-Grammatik. Sie können dabei die Grammatik MiniLispA oder MiniLispB im Sample Project als Ausgangspunkt nutzen und diese anpassen und vervollständigen. Erzeugen Sie mithilfe der Grammatik und ANTLR einen Lexer und Parser.

3. Führen Sie die semantische Analyse durch: Sind alle Symbole bekannt, passen die Scopes?

4.2 A5.2: Tree-Walking-Interpreter (5P)

Bauen Sie einen Tree-Walking-Interpreter in Ihr Projekt ein.

Realisieren Sie die eingebauten Funktionen `print` und `str` dabei als *native* Funktionen. Realisieren Sie `list`, `nth`, `head` und `tail` sowie `def`, `let`, `defn`, **`do`** und die Operatoren und die Kontrollstrukturen geeignet.

Lösen Sie die als "*syntactic sugar*" bezeichneten Ausdrücke auf und transformieren Sie den AST entsprechend: `(+ 1 2 3 4)` soll zu `(+ (+ (+ 1 2)3)4)` umgeformt werden. Analog für die anderen Operatoren der Sprache (Vergleiche, Arithmetik).

Achten Sie auf die Datentypen. Die Typen von Variablen etc. sind erst zur Laufzeit bekannt und müssen dann passen.

Lesen Sie den zu interpretierenden Code aus einer Datei ein.

Testen Sie Ihren Interpreter mit Ihren Beispielprogrammen.

4.3 A5.3: Auswirkungen der Grammatik auf den Interpreter (2P)

Sie haben sich vermutlich für eine der beiden Grammatiken (MiniLispA, MiniLispB) entschieden und auf der Basis Ihren Interpreter erstellt.

Welche Auswirkungen hat die Grammatik auf den Interpreter? Machen Sie ein Gedankenexperiment: Überlegen Sie, was Sie alles in Ihrer Implementierung ändern müssten, wenn Sie die jeweils andere Grammatik-Variante nutzen würden.

4.4 Bonus: Interaktiver Interpreter (3P)

Bauen Sie eine *REPL* ein, d.h. geben Sie nach dem Start des Interpreters einen Prompt aus und verarbeiten Sie die Eingaben interaktiv. Wie müssen Sie hier mit der Symboltabelle umgehen?

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9cb045f (readme: no more reference-links when formatting w/ pandoc, 2025-07-25)